Suggested Exercises

Difficulty: 🌱 warm-up · 🧩 intermediate · ⛰️ hard

Session 2 is about running, breaking and observing a plan that already exists. **Session 3** is about extending it and sending the change to someone else.

Most exercises run on `main`. The ones that do not are marked with a **branch:** line, and live on that branch of this repo:

```
git checkout <branch>
ob run benchmark.yaml --out-dir out_<branch> -c 2
```

The branches are named and described as if they were ordinary patches done with some purpose (that is how you normally get broken plans!). Reading the diff is allowed, but for most of them it will not be obvious what is wrong.

Nothing here needs more than the `sc-mix` dataset and running with one or two cores unless it says so. Answer sections will be published separately.

Session 2 — reading, running, breaking

A · ENVIRONMENTS AND RUNNING

1 · 🌱 First run, and observe where everything landed

Run the benchmark and browse the output folder (`out` by default).

- Where does snakemake store the conda environments? How many are there, and how big is each?
- What is in `out/.envs/`, `out/.logs/`, `out/.modules/` and `out/.metadata/`?
- Every leaf directory exists twice: once as `dataset_name-sc-mix` and once as `.6fbd9e19`. Which one is real, and why are there two?

---

2 · 🧩 Second run, without waiting for conda

Run again into a fresh folder: `ob run benchmark.yaml --out-dir NEWFOLDER`. Can you avoid paying for environment creation a second time? Use only standard UNIX tools.

- What exactly is the cache key?
- Which is safer, copying or symlinking, and why?

---

3 · 🧩 Read the failure, not the traceback

Delete `bioconductor-zellkonverter` from `envs/r_osta.yml` and rerun. Snakemake prints a 40-line `RuleException`. The actual R error is one line, in a different file.

Find it without scrolling the snakemake log.

---

4 · 🧩 Change one pin, count the environments

Bump `r-argparse` to a specific version in `envs/r_osta.yml`, rerun, then `du -sh out/.snakemake/conda/*/`.

You now have four environments totalling ≈7 GB, one of which nothing references. Which one, and how do you know?

---

5 · ⛰️ Containers instead of conda (Linux only)

Port these environments to Docker images and run the benchmark with apptainer instead.

- Can you quantify the virtualization overhead? Against what baseline?
- Which of the three environments is worth containerizing first?

---

6 · ⛰️ One file to install and run them all

Assuming you built images in exercise 5, write a single script using PEP 723 inline metadata that auto-installs `omnibenchmark` and its Python dependencies, then runs your benchmark plan.

`uv run mybench.py` with no venv, no README, no instructions. How much of the "Installing ob" section of the README does it replace?

---

7 · ⛰️ Predict (resources) before you run

Uncomment the full parameter grid for `osta-r`, and the first three datasets (`sc-mix`, `bel`, `cb`).

Before executing anything: **how many jobs?** Write the number down. Then:

```
ob run benchmark.yaml --dry
grep -c '^rule ' out/Snakefile
```

Now estimate wall-clock from the `*_performance.txt` files of your first run. Then decide whether to run it.

B · CHANGING THE PLAN

8 · 🌱 A sweep, from the editor

Use obeditor to add a small parameter sweep to one of the modules. Two or three values should be ok, not the full grid. Once you are happy with it, save and run it locally.

Does the YAML it produces make `ob validate plan` happy? Did you catch any errors?

---

9 · 🌱 The dev loop

Clone `pipelines-py` locally, point the module's `repository:` at your local path, change one line in the scanpy script, and rerun only that module:

```
ob run benchmark.yaml --dirty -m scanpy
```

- What does `-m` prune, and what does it keep?
- Why does `--dirty` exist as a separate flag rather than being implied by a local path?

---

10 · 🌱 `track-upstream`

branch: `track-upstream`

Let's have a look at the commit message on the last commit on this branch:

> *Track pipelines-r main while the OSCA fixes land*

This one gives itself away in the diff, on purpose. It is more about situations that you might encounter with your team while benchmarking.

- Run it. What does `ob` refuse to do, and what does it suggest instead?
- Run it again with `--unpinned` and let it succeed. Inspect `out_track-upstream/.metadata/manifest.json`. What exactly did you benchmark?
- Six months from now, someone tries to reproduce this result. What do they have, and what do they need?
- When *is* `--unpinned` the right flag?

---

11 · 🧩 A threaded entrypoint

Not a bug: a modification, and a prerequisite for exercise 18.

Fork `pipelines-r` and add `run_pinned.sh`:

```
#!/usr/bin/env bash
export OMP_NUM_THREADS=${OB_THREADS:-1} \
       OPENBLAS_NUM_THREADS=${OB_THREADS:-1} \
       MKL_NUM_THREADS=${OB_THREADS:-1}
exec ./run.R "$@"
```

Register it in the module's own `omnibenchmark.yaml`:

```
entrypoints:
  default: run.R
  pinned: run_pinned.sh
```

and select it from the plan without touching the module again:

```
repository:
  url: https://github.com/<you>/pipelines-r
  commit: <your commit>
  entrypoint: pinned
```

- `entrypoint:` takes a *key*, not a path. Why is that "indirection" worth having?
- What happens if you forget the `#!` line, or the `chmod +x`?
- Do the same for `pipelines-py`.

C · DEBUGGING: A FEW BROKEN BRANCHES

12 · 🧩 `csv-outputs`

branch: `csv-outputs`

As before, start from the message on the only commit on this branch:

> *Emit cluster assignments as CSV for downstream spreadsheet users*

Run it. The method modules print a clean log and exit successfully, and the benchmark still fails.

- Which rule failed, and at what point in its lifecycle?
- Nothing in the module's log is an error. Where is the actual complaint?
- What is the minimal fix, and how many places must change?

---

13 · 🧩 `rename-pca-output`

branch: `rename-pca-output`

Again, the commit message seems legit:

> *Rename PCA output id for consistency with the clustering_info naming*

But oh no!…

- This one never starts a job. At which stage of `ob run` does it die? validation, fetching, resolution, or execution?
- Compare with exercise 12: both are renames. Why does one fail before any work happens and the other only after?

---

14 · 🧩 `slim-inputs`

branch: `slim-inputs`

As a good benchmarker, start from the commit message:

> *Drop unused truth-count input from the methods stage*

Sounds good, no? The removed input really did look unused from the plan's point of view.

- Both method modules now fail, identically. What is the error?
- Explain the mechanism: what does an entry under a stage's `inputs:` actually create?
- The commit message asserts the input was unused. What evidence would have falsified that before committing?

---

15 · 🧩 `rename-params`

branch: `rename-params`

By now you are reading commit messages with a sane dose of skepticism:

> *Use a descriptive parameter name in the osta-r sweep*

- The `scanpy` method job completes (you can walk the output tree and check the logs). The `osta-r` one does not, and neither metrics job gets to run. Why does a rename in `benchmark.yaml` reach inside a module at all?
- Fix it two different ways. Which one would you prefer in review?
- How could you have caught this without running the benchmark?

D · MONITORING, PROFILING, VISUALISATION

16 · 🌱 Collect the performance numbers and plot them

Every job writes a `*_performance.txt` next to its outputs:

```
s   h:m:s    max_rss max_vms max_uss max_pss io_in   io_out  mean_load   cpu_time
```

- Gather them with `ob collect performance`. What does it key the rows on?
- Plot wall-clock and `max_rss` per rule. Which stage dominates, and is that what you expected?
- `mean_load` for `osta-r` was 41.7 on a run given one core. What does that number mean, and is it a problem?

---

17 · 🧩 Profile a run with `denet`

branch: `profiling` — this is `main` plus a `scripts/` folder; the plan itself is unchanged, so everything below also works on `main` if you would rather assemble the commands yourself.

`denet` samples a process tree over time, the thing `*_performance.txt` summarises into one row. It is packaged in the lab channel:

```
conda create -n prof -c https://repo.prefix.dev/almost-conductor -c conda-forge denet
denet --help
```

- Profile a single module's entrypoint, then the whole `ob run`. What does the second view show that the first cannot?

- Plot RSS over time for `osta-r`. Where does the 3.1 GB peak actually happen — loading, PCA, or clustering?
- Snakemake's `max_rss` is a *sample* maximum too. Do the two agree?

Both runs are scripted, so you do not have to assemble the denet invocation by hand — read them first, they are about forty lines each:

```
git checkout profiling
scripts/profile.sh                        # or: scripts/profile.sh OUT_DIR
python3 scripts/aggregate.py out_prof.prof/*.jsonl
```

`profile.sh OUT_DIR` writes the whole-DAG run to `OUT_DIR`, the single-module run to `OUT_DIR_m` and the traces to `OUT_DIR.prof/`. The two runs deliberately do **not** share an out-dir — if they did, the second would find everything cached and its trace would show nothing.

That is two full benchmark runs. If you already have an out-dir built, point `CONDA_PREFIX_DIR` at its conda folder and snakemake reuses those environments instead of resolving 5 GB again:

```
CONDA_PREFIX_DIR=$PWD/out/.snakemake/conda scripts/profile.sh
```

`denet stats FILE` prints the summary for a trace on its own; `aggregate.py` only adds the curve `denet` will not draw, as plain SVG so you do not have to install matplotlib into the `prof` env.

For the *entrypoint* half of the first bullet, do not wrap `ob run` — wrap the module. `scripts/run_profiled.sh` on that branch is a copy-me wrapper; put it in your exercise 11 fork and register it beside the entrypoint you added there:

```
entrypoints:
  default: run.R
  pinned: run_pinned.sh
  profiled: run_profiled.sh
```

Then select it from the plan (`entrypoint: profiled`) and run with `DENET` set to an absolute path:

```
DENET=$(command -v denet) ob run benchmark.yaml --out-dir out_ep -m osta-r -c 1
```

The trace lands in the job's own output directory, under the same parameter hash as everything else it produced.

- Why does the wrapper need `$DENET` as an absolute path rather than just calling `denet`? (Exercise 22 has the same problem with a different program.)
- With `DENET` unset the wrapper is the default entrypoint. Why is that worth the two extra lines?

---

18 · 🧩 Was the last comparison fair?

Baseline for this benchmark under `--cores 1`:

rule	wall (s)	cpu_time	mean_load
`osta-r`	34.67	14.49	41.7
`scanpy`	29.56	15.13	50.8

Snakemake was given one core — but `--cores` does not stop a job from spawning threads, and neither module declares a thread budget. On `sc-mix` neither actually went above one core (both loads are under 100; see exercise 16), yet the two did not get the same share of it.

- Rerun both under the `pinned` entrypoint from exercise 11 with `OB_THREADS=1`, then `=4`. Report wall-clock **and** `cpu_time / wall` for each.

- Under which conditions is “scanpy is faster than OSCA” a statement about the pipelines rather than about BLAS?
- `--cores` bounds how many *jobs* snakemake starts, not how many threads each job spawns. Where should that second budget be enforced? the plan, the module, or the environment?

---

19 · 🧩 Make the thread count provenance

Extend `run_pinned.sh` to write `$OUTPUT_DIR/env.json` containing `nproc`, the three `*_NUM_THREADS` values, and the BLAS line from R’s `sessionInfo()`. Declare it as a module-level `outputs:` entry — modules may emit more than their stage’s shared contract (`datasets-r.hvgs.tsv` in your output tree is an existing example).

“If it is not in the output tree, it did not happen.”

Session 3 — extending and contributing

E · PLAN-LEVEL TOPOLOGY

20 · 🧩 Resources, per stage and per module

Add `resources: {cores: 4, mem_mb: 8000}` at the `methods` stage level, then override it for `metrics-py` alone — that module peaks at 146 MB RSS and runs in 1.5 s.

- Verify in the generated `out/Snakefile`, not by hoping.
- Module-level `resources` overrides stage-level. When does this stop being cosmetic? (Hint: the SLURM executor.)

---

21 · 🧩 Prune the grid with `exclude:`

The `1.3m` dataset does not fit through the R pipelines. Keep `osta-r` off it — without forking the plan, and without touching the Python "path".

First problem: `exclude:` names **module ids**, and `1.3m` is a *parameter value* of `datasets-r`, not a module. So split it out — add a second module to the `datasets` stage, same repo and commit as `datasets-r`, with `id: datasets-r-big` and `dataset_name: ["1.3m"]` — then put `exclude: [datasets-r-big]` on `osta-r`.

Stay on `-d` (`--dry`) throughout: nobody wants 1.3M cells running on their laptop. The whole exercise is about the job count in the generated `Snakefile`, which `-d` gives you without running anything.

The field is **symmetric** (declare it on either module — same result), **lineage-wide** (it prunes any path where both ids appear, however many stages apart), and **OR'd** over the list.

- How many rules before, how many after, and why is the drop 2 rather than 1?
- Move the `exclude:` onto `datasets-r-big` instead, pointing back at `osta-r`. Same Snakefile? Should it be?
- Now write `exclude: [scanpy]` on `osta-r` — two modules in the *same* stage. Predict the count, then check. Why does `ob validate plan` still say ✅?
- Try `exclude: ["1.3m"]` — the parameter value rather than a module id. What does validation say, and what did the plan actually do?
- Add `exclude: [datasets-r-big, datasets-r]`, so `osta-r` excludes both dataset modules. Now validation *does* complain. What is it protecting you from?
- Where would you put "this module needs a GPU"? Not here. Say why, then look at `resources:` from exercise 20 and say what it does and does not buy you.

The grid is a Cartesian product until you say otherwise. `exclude:` is how you say otherwise without forking the plan — as long as the thing you want to exclude is a module.

F · NEW STAGES AND MODULES

22 · 🧩 Turn the collector back on

worked solution on branch: `collector` — try it yourself first; that branch is `main` with this exercise already done, for comparing against or for falling back on if you run out of time.

Uncomment the `metric_collectors:` block at the top of `benchmark.yaml`. It references `r_apptainer`, a software environment this plan does not define, and `pipelines-report`, a module repo you have not run before. Neither is as much work as it looks.

- What does validation say? Fix one complaint at a time.
- Define the missing environment as a conda one (`envs/` has three worked examples) and get the collector to run under it.
- Its outputs are declared as `report/metrics.tsv`, `report/timings.tsv` and `report/plots.html`. Where do those land, and why is that not under a dataset/parameter hash directory like everything else?
- A collector fans **in** over every branch of the DAG; a stage module maps **over** one branch. Why does that difference need its own top-level key rather than being another stage?

Do not port the apptainer image. Read the module's `run.R` and `plots.Rmd` and list what they actually load — that is the whole job, and it is shorter than it looks.

When it runs: open a PR against `main` of this plan with the uncommented block and your environment file. That is a **plan** PR, unlike exercise 23's, which is against a module — the two get reviewed by different people and mean different things, so it is worth doing one of each.

- What does your diff have to contain for someone else to reproduce your report from a clean checkout?
- Your `envs/` file pins versions. Whose job is it to keep those pins current, and what breaks in a year if nobody does?

---

23 · 🧩 Add a new metric — and send a PR

Fork `metrics`, add one metric to `run.py` (V-measure, Fowlkes–Mallows, anything the truth labels support), emit it into the existing `{dataset}.metrics.json`, and open a PR against the module repo.

The `metrics` module here is a trimmed version of the one in the original scRNA benchmark (Billato et al., 10.1101/2025.10.28.681564). The lazy route is the intended one: find the metric you want in the upstream modules and reuse it, rather than reimplementing it from the paper.

- Does adding a key to an existing output file need a plan change? Why not?
- What *would* need one?
- Pick a metric that upstream computes and this plan dropped. Do the numbers match theirs on `sc-mix`? If not, is that the metric, the pipeline, or the parameters?
- Your PR is against the module. Who has to do what before the published benchmark reflects your metric?

---

24 · 🧩 Add a no-op stage — and send a PR

Add a stage between `methods` and `metrics` that passes its inputs straight through. Then open a PR against **this** repo with the plan change.

The point is the wiring, not the computation: a stage that does nothing still has to declare inputs, outputs, ids and paths that line up on both sides. Count how many things you had to get right for a stage that computes nothing.

---

25 · ⛰️ A permutation baseline

Add a module to `methods` that shuffles the truth labels and emits them as its clustering. Every metric in the benchmark should now have a floor.

- Where does each metric actually land? Is any of them *not* near chance?
- How many permutations before the floor is a distribution rather than a number, and where do you put the seed so the run stays reproducible?
- Should a baseline be a module in `methods`, or its own stage? Defend it.

G · CI AND VALIDATION

26 · 🧩 Fork the plan and give it CI

Fork this repo and add a GitHub Actions workflow that runs `ob validate plan` on every push. Then ask a facilitator to open a PR against your fork — they will send you one of the Session 2 breakages, maybe.

Does your CI catch it? Which of exercises 12–15 *could* it have caught, even in principle?

---

27 · ⛰️ A validator that would have caught more

Extend the CI to catch what `ob validate plan` cannot.

- Every stage input id is provided by some upstream stage output id (exercise 13, before anyone runs anything).
- Every module `repository.commit` is a full-length immutable ref, not a branch (exercise 10).
- Every `parameters:` value is a list. Both forms ship in `main` today (`filter: ["manual"]` for `osta-r`, `filter: "manual"` for `scanpy`) and the benchmark runs green — so is this a check, or a style preference? Decide before you write it.

Then decide which of these belong in `ob` itself rather than in your CI, and open an issue upstream for the best one.

SUGGESTED ORDER

Session 2, block C is the core: start at **13** and **12** back to back — same edit shape, opposite failure timing — then **14**, then **15**.

11 → 18 → 19 is one continuous thread and only makes sense in that order.